Project 2: Linear Programming - Simplex

Team Information

Team Name: **I'm Only Kenny (I'm OK)**
Team Members: **Kenneth O'Dell**

Name: Project 2 – Linear Programming - Simplex

Date: 2026-10-18

Exemption to work alone was confirmed via email.

Public Link to Demo and Summary Video

Link to GitHub with source code:

https://github.com/ko222uky/lp-simplex

Introduction

The main objective of this project is to compare linear programming (LP) problems of increasing size in terms of decision variables (n) and constraints (m) and analyze the behavior of an implementation of the simplex algorithm.

LP Problems

The following description of the LP problem is informed heavily by Huangfu and Hall's "Parallelizing the dual revised simplex method" (2018). The general form of the LP problem is given as:

- minimize $f = c^T x$
- subject to $Ax = 0$ and $l \leq x \leq u$

The matrix A, which is the m x n coefficient matrix, is such that it is guaranteed to have m linearly independent columns from which we obtain our invertible basis *B,* with the rest forming non-basic variables *N (p.29,* Papadimitriou 1998). Therefore, we have the form,

- $Ax = Bx_B + Nx_N = 0,$

and cost vectors may be partitioned into basic and nonbasic costs:

- $f = c^T_B\ x_B + c^T_N\ x_N$

It should also be known that the basic variables are bounded, such that for all basic variables $\mathbf{x_B}$, if it is bounded by $\mathbf{l_B}$ from below and $\mathbf{u_B}$ from above, then we say that the basis is primal feasible; otherwise, it has primal infeasibility if a given variable x_i violates these bounds. For the non-basic variables in *N*, if it holds for all x_j in *N* that:

(1) cost of $x_j \geq 0$ if $x_j = l_j$ (lower bound for index j)
and
(2) cost of $x_j \leq 0$ if $x_j = u_j$ (upper bound for index j)

Then the basis is dual feasible. This basically means that moving the non-basic variable won't improve the cost, since the cost cannot improve between l_j and u_j, and the optimal cost is at $x_j = u_j$ for the given basis B.

Problem Instances

The original problem comes from the lecture slides, where A, b and c are defined as follows:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & -1 & -1 \\ -1 & -5 & 2 & 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{c}^\top = [7 \quad 4 \quad 6 \quad 1]$$

To increase the number of variables *n* and constraints *m*, additions were made to A, b and c. First, a random matrix A2 was constructed with density = 0.5 and coefficients drawn from integers in the range [-20, 20] / 0 (excluding 0). A block diagonal matrix A_{block} was then constructed as follows:

$$A_{block} = [A, 0$$
$$0, A2]$$

This creates a larger problem but the variables of A and A2 do not interact. Thus, I defined two more matrices $A_{topright}$ and $A_{bottomleft}$ such that I could guarantee feasible solutions. The matrix $A_{topright}$ is an m x $n_{additional}$ matrix constructed from random positive integers in the range [1, 20], also with density 0.5. The $A_{bottomleft}$ is an n x $m_{additional}$ matrix drawn from *negative* integers in the range [-20, -1]. Thus, the final A_{block} is defined as:

$$A_{full} = [A, A_{topright}$$
$$A_{bottomleft}, A2]$$

To extend the vectors b and c, I simply defined b2 and c2 based on A2 and then redefined b_{full} = concat(b, b2) and c_{full} = concat(c, c2).[1] Thus, the extended problems, using the original lecture LP as a base, were a set of new objects A_{full}, b_{full} and c_{full}. The values for the additional n and m were set so that I tested all combinations of n values of 4, 10, 20, 30, 40 and 50 and m values of 2, 6, 10 and 14.

All problems are randomly generated but the seed is fixed, allowing for reproducibility.

Methods

Dual Simplex Algorithm (HiGHS)

The current code utilizes the serial (not parallelized version) of the simplex algorithm as described in Huangfu and Hall's "Parallelizing the dual revised simplex method" (2018).[2] Given the description of the LP problem in the introduction, the dual simplex method aims to maintain dual feasibility while improving a solution towards primal feasibility. Huang and Hall give three main steps: (1) choose the row, (2) choose the column, and (3) update the values. The following describes the non-parallelized version:

```
Function dual_simplex:
      Choose index p of a primal infeasible variable
      If p not found:
            LP is solved
      For each q in N:
            // choosing q requires the ratio test and computing
```

[1] The actual definition from my Julia code is:

```
A2 = sprand(rng, additional_m, additional_n, density,
     (r, k) -> Float64.(rand(r, [-20:-1; 1:20], k)))
x0 = rand(rng, 1:20, additional_n)
y0 = rand(rng, 1:20, additional_m)
b2 = A2 * x0 .- rand(rng, 1:20, additional_m)
c2 = A2' * y0 .+ rand(rng, 1:20, additional_n)
```

Then, b2 needed to be redefined again as:

```
x_lec = [2.0, 0.0, 0.0, 1.0] # feasible for orig lecture vars
b2 = b2 .+ B * x_lec
```

This required some trial and error on my part.

[2] This is defined as strategy 1 (https://ergo-code.github.io/HiGHS/dev/options/definitions/#option-simplex-strategy).

```
        // a reduced coefficient matrix A hat.
        Choose q to enter basis such that the rest of N
        remains dual feasible
    Update basis B by interchanging p and q
    Update values (primal and dual variables)
```

Thus, the idea is to maintain dual feasibility and then improve our selection of variables leaving the basis until we have primal and dual feasibility (at which point we have an optimum).

Code Execution

This project was implemented with Julia with two main dependencies, **JuMP**, which is a "modeling language and collection of supporting packages for mathematical optimization in Julia" (JuMP | JuMP is a modeling language and supporting packages for mathematical optimization in Julia.). This, along with the right supporting package, provides macros that allow the user to easily model an LP problem by specifying variables, constraints, and the objective. The supporting package used with **JuMP.jl** was **HiGHS.jl**, which is a wrapper for HiGHS (HiGHS - High-performance parallel linear optimization software). Thus, you can use the JuMP model with the HiGHS optimizer (similar to **linprog** in **SciPy**), the model parameters can be defined as such so that a simplex is used, with either the primal or dual approach (defined by `simplex_strategy`):

```
m, n = size(A)
model = Model(HiGHS.Optimizer)
set_silent(model)
set_attribute(model, "solver", "simplex")
set_attribute(model, "simplex_strategy",
strategy)
set_attribute(model, "presolve", presolve)
set_attribute(model, "threads", threads)
set_attribute(model, "time_limit", time_limit)
```

This is the core model. Then, the LP is defined with macros:

```
@variable(model, x[1:n] >= 0)
@constraint(model, A * x .>= b)
@objective(model, Min, dot(c, x))
```

The problem is then solved using `optimize!(model)`, and the results can be obtained by passing the model to various JuMP functions (e.g., `simplex_iterations`, `solve_time`).[3] The presolve parameter was not used for these experiments. The code is executed with a single line in the terminal: `julia --project=. main.jl`

[3] The '!' suffix on a function indicates that the function mutates its inputs in-place, so the model object has its attributes set to the optimization results.

Results

For each experiment, five replicates were run to obtain statistics. This summary is shown in **Table 1** and **Figures 1, 2**.

Table 1: Effects of number of decision variables (n) and constraints (m) on the runtime and number of iterations. All extended problems were generated with the same matrix density of 0.5.

n	m	Matrix	Mean runtime	Runtime std	pivots
4	2	0.5	0.000168	4.77E-05	3
10	2	0.5	0.000153	6.47E-05	2
20	2	0.5	0.000246	2.82E-05	3
30	2	0.5	0.000235	2.81E-05	3
40	2	0.5	0.000238	5.38E-05	2
50	2	0.5	0.000255	4.53E-05	1
4	6	0.5	0.000184	5.82E-05	3
10	6	0.5	0.000212	2.13E-05	9
20	6	0.5	0.000225	4.28E-05	7
30	6	0.5	0.000278	5.49E-05	6
40	6	0.5	0.000329	6.49E-05	14
50	6	0.5	0.000365	8.41E-05	11
4	10	0.5	0.000239	2.17E-05	3
10	10	0.5	0.000277	5.62E-05	10
20	10	0.5	0.000276	5.99E-05	15
30	10	0.5	0.000419	7.56E-05	31
40	10	0.5	0.000381	0.000104	25
50	10	0.5	0.000385	0.000112	38
4	14	0.5	0.000167	4.43E-05	3
10	14	0.5	0.00019	4.62E-05	7
20	14	0.5	0.000344	8.28E-05	25
30	14	0.5	0.000434	0.000119	37
40	14	0.5	0.000445	5.28E-05	30
50	14	0.5	0.00053	0.000141	42

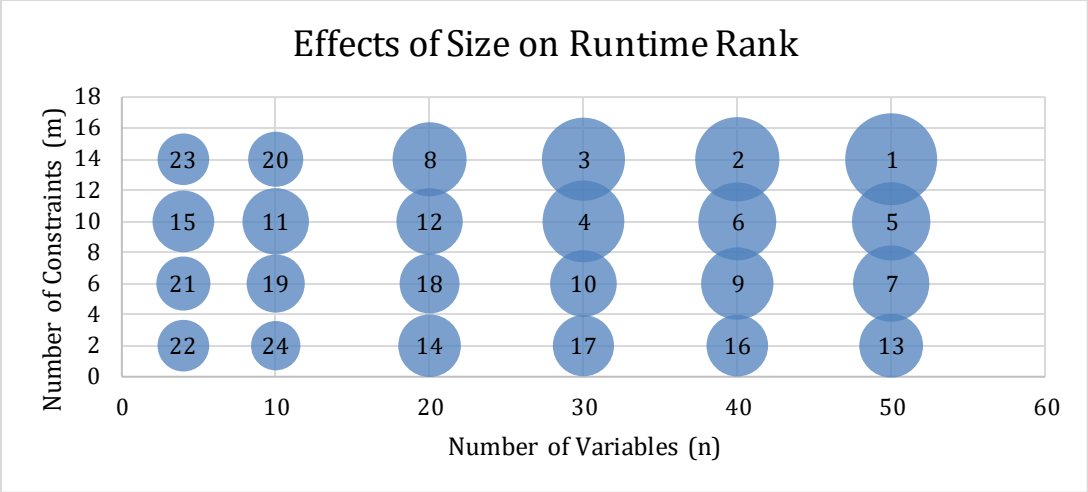


Figure 1: Effects of problem size on the runtime, where each problem was ranked based on its runtime. A runtime rank of 1 means that problem had the highest runtime (slowest). Thus, the size of the bubble reflects the problems runtime. Note that both n and m are positively correlated with increasing runtime. The slowest runtime was for the problem with m=14, n=50, whereas the fastest runtime was for the problems with m=2.

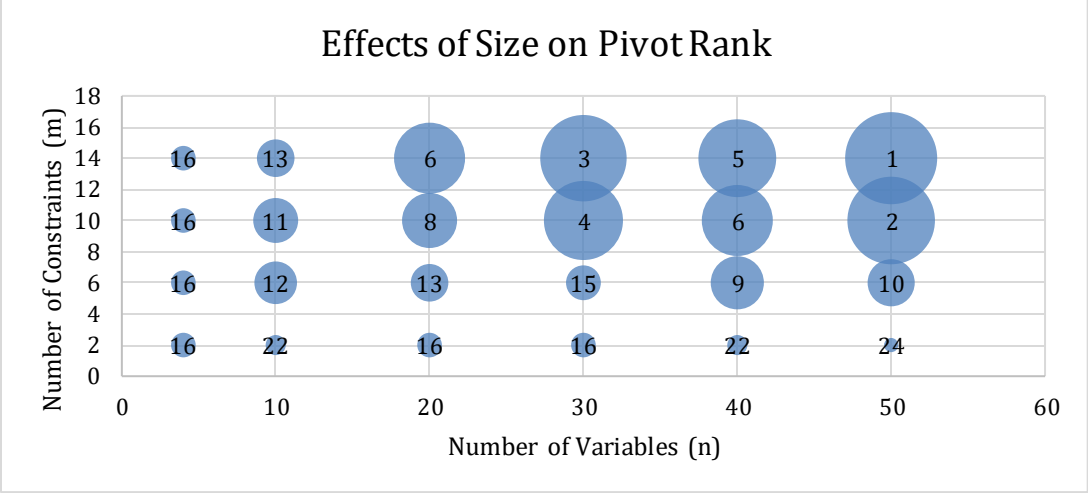


Figure 2: Corresponding effects of problem size on the number of pivots, where each problem was ranked by the number of required pivots. Differences were more pronounced than runtime differences. Note that fixing one variable of problem size (either n or m) and increasing the other did not result in significantly more pivots (compare data at m = 2 with increase n, or n = 4 with increasing m).

Experimental Analysis

General Observations on Runtimes

In general, increasing *n* resulted in an increase in runtime (**Figure 1**, following the data horizontally in the graph), and increasing *m* mostly impacted runtime at higher n's

(following the data vertically). The relation between runtime and number of pivots has a positive relationship, as can be seen in the results in **Table A 1** in the appendix.

General Observations on Number of Pivots

The number of pivots were clearly related to the runtime, which can be seen when comparing **Figures 1** and **2.** The most pronounced increase in computation (whether one takes the runtime or pivot number) were obtained when both *m* and *n* were increased. Again, more pivots were found in the problem with m=14, n=50. Thus, it seems that at a sufficient problem size, effects on the number of pivots appear. At n=50, increasing m drastically increases the number of pivots, but this is not the case at n=4.

Overall Speed

The speed of the experiments were very fast and differences were negligible from a practical standpoint. This could be due to how I defined the problems, all of which were strictly feasible and were guaranteed to yield an optimal, non-degenerate result (**Table A 1**). To be sure, the algorithm took longer to process for problems with the most variables and constraints.

Degeneracy

The results from the HiGHS optimizer did not find any degenerate solutions based on a simple Boolean check (see **Table A 1**). This could be because the original "seed" problem given by the lecture example is strictly feasible, and this problem was expanded in such a way so that the original solution space was not "cut into" by the added constraints. That is, the way I defined the newer LPs were such that the newer problems were strictly feasible in both primal and dual sense since the blocks added to matrix **A** were not coupled to the original lecture constraints. Thus, we were (1) guaranteed to find a feasible solution and (2) the solution we find would not be degenerate.

Conclusion

In summary, the performance of simplex and its results obtained did show a positive increase in runtime and pivot numbers when the decision variables and constraints were increased sufficiently. Adding more constraints didn't always increase the number of pivots or runtime since the problems were defined such that constraints didn't cut into the original feasible space (see examples where n=2 and m increase in **Table A** 1). Perhaps the most informative lesson learned pertained to how the problem instances were generated. The approach I took aimed to obtain feasible problem instances but perhaps I should have included malformed problems, infeasible problems, and non-strictly feasible problems to explore more thoroughly the algorithm's performance and behavior.

Challenges in Generating Problems

One initial challenge was defining the problem instances. The original implementation using A_{full} resulted in primal infeasible solutions obtained since the following constraint for the

bottom rows could be violated in some randomly generated problem instances, resulting in no optimal solutions obtained:

$$A_{bottomleft} * x + A2 * x_{additional} >= b2$$

This is because $A_{bottomleft}$ is negative, which drags the left side of the equation down possibly below the lower bound. However, changing the matrix $A_{bottomleft}$ to positive resulted in dual infeasible problems, which would require fixing how I extended the cost function. Thus, to fix this, I decided to define b2 differently. Thus, let's have a new variable b_{fix}, which is defined using b2 and adding $A_{bottomleft} * x_{optimal}$, where $x_{optimal} = [2, 0, 0, 1]$ for this problem. The value $x_{optimal}$ was chosen since we know $x_{optimal}$ is obviously primal and dual feasible. It can be shown that the obtained matrix A_{full} using b_{fix} would be guaranteed to give us a feasible problem since:

$$b_{fix} = b2 + A_{bottomleft} * x_{optimal}$$

and thus, we have the constraints for the bottom rows of A_{full}:

$$A_{bottomleft} * x_{optimal} + A2 * x_{additional} >= b_{fix}$$

Substituting for b_{fix}, we obtain:

$$A_{bottomleft} * x_{optimal} + A2 * x_{additional} >= b2 + A_{bottomleft} * x_{optimal}$$

The $A_{bottomleft} * x_{optimal}$ terms cancel out, giving us:

$$A2 * x_{additional} >= b2$$

This is now guaranteed to be satisfied since we defined b2 directly from A2.

However, one caveat of this definition is that all of my LPs were strictly feasible and guaranteed to yield an optimal solution. Thus, the results may not be interesting in terms of analyzing *when* simplex fails. This also means that additional experiments would be needed to analyze the runtime and number of pivots for problem instances that are infeasible (no solution is possible) or non-strictly feasible (so degenerate solutions exist). Furthermore, problems could be defined such that they were only primal feasible or only dual feasible. These might've allowed more opportunities to explore the algorithm's performance.

In conclusion, I've demonstrated how to define LPs of varying sizes, guarantee their strict feasibility, obtain optimal solutions using APIs available through Julia, tabulate results for LP Simplex experiments, and analyze the behavior of the algorithm based on the LP size in the context of how the LPs were defined. Future work may expand this by analyzing and considering more thoroughly the way that the LP construction and definition impacts the Simplex behavior and performance.

References

Kwon, C., 2019. *Julia programming for operations research*. Changhyun Kwon. **Covers a detailed implementation of the Simplex algorithm in Julia using the Tableau method. Also covers examples of using JuMP and HiGHS.**

Huangfu, Q.H.J.A.J. and Hall, J.J., 2018. Parallelizing the dual revised simplex method. *Mathematical programming computation*, *10*(1), pp.119-142. **Source paper for the pseudocode and implementation of the HiGHS simplex.**

Papadimitriou, C.H. and Steiglitz, K., 1998. *Combinatorial optimization: algorithms and complexity*. Courier Corporation. **Provides the theoretical foundation for Simplex.**

Appendix

All Tabulated Results

The complete tabulated results are shown here.

*Table A 1: Complete tabulated results for strategy=1 of the HiGHS optimizer. The solution for the higher values of **n** is too large to fit in this report.*

n	m	rep	density	status	time	iters	obj	degenerate
4	2	1	0.5	OPTIMAL	0.000202	3	15	FALSE
4	2	2	0.5	OPTIMAL	0.000231	3	15	FALSE
4	2	3	0.5	OPTIMAL	0.000156	3	15	FALSE
4	2	4	0.5	OPTIMAL	0.000121	3	15	FALSE
4	2	5	0.5	OPTIMAL	0.000128	3	15	FALSE
10	2	1	0.5	OPTIMAL	8.92E-05	2	0.333333	FALSE
10	2	2	0.5	OPTIMAL	0.000119	2	0.333333	FALSE
10	2	3	0.5	OPTIMAL	0.000145	2	0.333333	FALSE
10	2	4	0.5	OPTIMAL	0.00015	2	0.333333	FALSE
10	2	5	0.5	OPTIMAL	0.00026	2	0.333333	FALSE
20	2	1	0.5	OPTIMAL	0.000207	3	1.583333	FALSE
20	2	2	0.5	OPTIMAL	0.000286	3	1.583333	FALSE
20	2	3	0.5	OPTIMAL	0.000252	3	1.583333	FALSE
20	2	4	0.5	OPTIMAL	0.000244	3	1.583333	FALSE
20	2	5	0.5	OPTIMAL	0.000241	3	1.583333	FALSE
30	2	1	0.5	OPTIMAL	0.000259	3	0.445833	FALSE
30	2	2	0.5	OPTIMAL	0.000223	3	0.445833	FALSE
30	2	3	0.5	OPTIMAL	0.000252	3	0.445833	FALSE
30	2	4	0.5	OPTIMAL	0.000191	3	0.445833	FALSE
30	2	5	0.5	OPTIMAL	0.000249	3	0.445833	FALSE
40	2	1	0.5	OPTIMAL	0.000219	2	0.204545	FALSE
40	2	2	0.5	OPTIMAL	0.000227	2	0.204545	FALSE
40	2	3	0.5	OPTIMAL	0.000222	2	0.204545	FALSE
40	2	4	0.5	OPTIMAL	0.000191	2	0.204545	FALSE
40	2	5	0.5	OPTIMAL	0.000331	2	0.204545	FALSE
50	2	1	0.5	OPTIMAL	0.000245	1	0.111111	FALSE
50	2	2	0.5	OPTIMAL	0.000284	1	0.111111	FALSE
50	2	3	0.5	OPTIMAL	0.000193	1	0.111111	FALSE
50	2	4	0.5	OPTIMAL	0.000312	1	0.111111	FALSE
50	2	5	0.5	OPTIMAL	0.000243	1	0.111111	FALSE
4	6	1	0.5	OPTIMAL	0.000168	3	15	FALSE
4	6	2	0.5	OPTIMAL	0.000126	3	15	FALSE
4	6	3	0.5	OPTIMAL	0.00018	3	15	FALSE
4	6	4	0.5	OPTIMAL	0.000281	3	15	FALSE

4	6	5	0.5	OPTIMAL	0.000163	3	15	FALSE
10	6	1	0.5	OPTIMAL	0.000238	9	1924.328	FALSE
10	6	2	0.5	OPTIMAL	0.00023	9	1924.328	FALSE
10	6	3	0.5	OPTIMAL	0.000204	9	1924.328	FALSE
10	6	4	0.5	OPTIMAL	0.000187	9	1924.328	FALSE
10	6	5	0.5	OPTIMAL	0.000202	9	1924.328	FALSE
20	6	1	0.5	OPTIMAL	0.000194	7	-570.742	FALSE
20	6	2	0.5	OPTIMAL	0.000202	7	-570.742	FALSE
20	6	3	0.5	OPTIMAL	0.000271	7	-570.742	FALSE
20	6	4	0.5	OPTIMAL	0.000187	7	-570.742	FALSE
20	6	5	0.5	OPTIMAL	0.000273	7	-570.742	FALSE
30	6	1	0.5	OPTIMAL	0.000255	6	-15914.4	FALSE
30	6	2	0.5	OPTIMAL	0.000345	6	-15914.4	FALSE
30	6	3	0.5	OPTIMAL	0.00022	6	-15914.4	FALSE
30	6	4	0.5	OPTIMAL	0.000328	6	-15914.4	FALSE
30	6	5	0.5	OPTIMAL	0.000243	6	-15914.4	FALSE
40	6	1	0.5	OPTIMAL	0.000323	14	-4864.36	FALSE
40	6	2	0.5	OPTIMAL	0.000403	14	-4864.36	FALSE
40	6	3	0.5	OPTIMAL	0.00036	14	-4864.36	FALSE
40	6	4	0.5	OPTIMAL	0.00033	14	-4864.36	FALSE
40	6	5	0.5	OPTIMAL	0.000227	14	-4864.36	FALSE
50	6	1	0.5	OPTIMAL	0.000305	11	-9784.52	FALSE
50	6	2	0.5	OPTIMAL	0.000314	11	-9784.52	FALSE
50	6	3	0.5	OPTIMAL	0.000373	11	-9784.52	FALSE
50	6	4	0.5	OPTIMAL	0.000508	11	-9784.52	FALSE
50	6	5	0.5	OPTIMAL	0.000325	11	-9784.52	FALSE
4	10	1	0.5	OPTIMAL	0.000264	3	15	FALSE
4	10	2	0.5	OPTIMAL	0.000236	3	15	FALSE
4	10	3	0.5	OPTIMAL	0.000218	3	15	FALSE
4	10	4	0.5	OPTIMAL	0.000218	3	15	FALSE
4	10	5	0.5	OPTIMAL	0.000258	3	15	FALSE
10	10	1	0.5	OPTIMAL	0.000312	10	7538.988	FALSE
10	10	2	0.5	OPTIMAL	0.00026	10	7538.988	FALSE
10	10	3	0.5	OPTIMAL	0.000333	10	7538.988	FALSE
10	10	4	0.5	OPTIMAL	0.000293	10	7538.988	FALSE
10	10	5	0.5	OPTIMAL	0.000189	10	7538.988	FALSE
20	10	1	0.5	OPTIMAL	0.000317	15	-12253.2	FALSE
20	10	2	0.5	OPTIMAL	0.000329	15	-12253.2	FALSE
20	10	3	0.5	OPTIMAL	0.000283	15	-12253.2	FALSE
20	10	4	0.5	OPTIMAL	0.000274	15	-12253.2	FALSE
20	10	5	0.5	OPTIMAL	0.000177	15	-12253.2	FALSE

30	10	1	0.5	OPTIMAL	0.000511	31	-27090.7	FALSE
30	10	2	0.5	OPTIMAL	0.000361	31	-27090.7	FALSE
30	10	3	0.5	OPTIMAL	0.000327	31	-27090.7	FALSE
30	10	4	0.5	OPTIMAL	0.000469	31	-27090.7	FALSE
30	10	5	0.5	OPTIMAL	0.000429	31	-27090.7	FALSE
40	10	1	0.5	OPTIMAL	0.000299	25	-40257.9	FALSE
40	10	2	0.5	OPTIMAL	0.000513	25	-40257.9	FALSE
40	10	3	0.5	OPTIMAL	0.000262	25	-40257.9	FALSE
40	10	4	0.5	OPTIMAL	0.000377	25	-40257.9	FALSE
40	10	5	0.5	OPTIMAL	0.000453	25	-40257.9	FALSE
50	10	1	0.5	OPTIMAL	0.000574	38	17031.57	FALSE
50	10	2	0.5	OPTIMAL	0.000325	38	17031.57	FALSE
50	10	3	0.5	OPTIMAL	0.000285	38	17031.57	FALSE
50	10	4	0.5	OPTIMAL	0.00036	38	17031.57	FALSE
50	10	5	0.5	OPTIMAL	0.000381	38	17031.57	FALSE
4	14	1	0.5	OPTIMAL	0.000139	3	15	FALSE
4	14	2	0.5	OPTIMAL	0.000233	3	15	FALSE
4	14	3	0.5	OPTIMAL	0.000177	3	15	FALSE
4	14	4	0.5	OPTIMAL	0.000171	3	15	FALSE
4	14	5	0.5	OPTIMAL	0.000116	3	15	FALSE
10	14	1	0.5	OPTIMAL	0.000185	7	-14649.7	FALSE
10	14	2	0.5	OPTIMAL	0.000146	7	-14649.7	FALSE
10	14	3	0.5	OPTIMAL	0.000176	7	-14649.7	FALSE
10	14	4	0.5	OPTIMAL	0.000269	7	-14649.7	FALSE
10	14	5	0.5	OPTIMAL	0.000176	7	-14649.7	FALSE
20	14	1	0.5	OPTIMAL	0.000374	25	-17028.4	FALSE
20	14	2	0.5	OPTIMAL	0.000453	25	-17028.4	FALSE
20	14	3	0.5	OPTIMAL	0.000324	25	-17028.4	FALSE
20	14	4	0.5	OPTIMAL	0.000342	25	-17028.4	FALSE
20	14	5	0.5	OPTIMAL	0.000225	25	-17028.4	FALSE
30	14	1	0.5	OPTIMAL	0.000296	37	-9466.88	FALSE
30	14	2	0.5	OPTIMAL	0.000319	37	-9466.88	FALSE
30	14	3	0.5	OPTIMAL	0.000509	37	-9466.88	FALSE
30	14	4	0.5	OPTIMAL	0.000565	37	-9466.88	FALSE
30	14	5	0.5	OPTIMAL	0.000479	37	-9466.88	FALSE
40	14	1	0.5	OPTIMAL	0.000501	30	-11629.4	FALSE
40	14	2	0.5	OPTIMAL	0.00046	30	-11629.4	FALSE
40	14	3	0.5	OPTIMAL	0.000445	30	-11629.4	FALSE
40	14	4	0.5	OPTIMAL	0.000358	30	-11629.4	FALSE
40	14	5	0.5	OPTIMAL	0.000461	30	-11629.4	FALSE
50	14	1	0.5	OPTIMAL	0.000493	42	-57599.8	FALSE

50	14	2	0.5	OPTIMAL	0.000689	42	-57599.8	FALSE
50	14	3	0.5	OPTIMAL	0.000651	42	-57599.8	FALSE
50	14	4	0.5	OPTIMAL	0.000476	42	-57599.8	FALSE
50	14	5	0.5	OPTIMAL	0.000343	42	-57599.8	FALSE